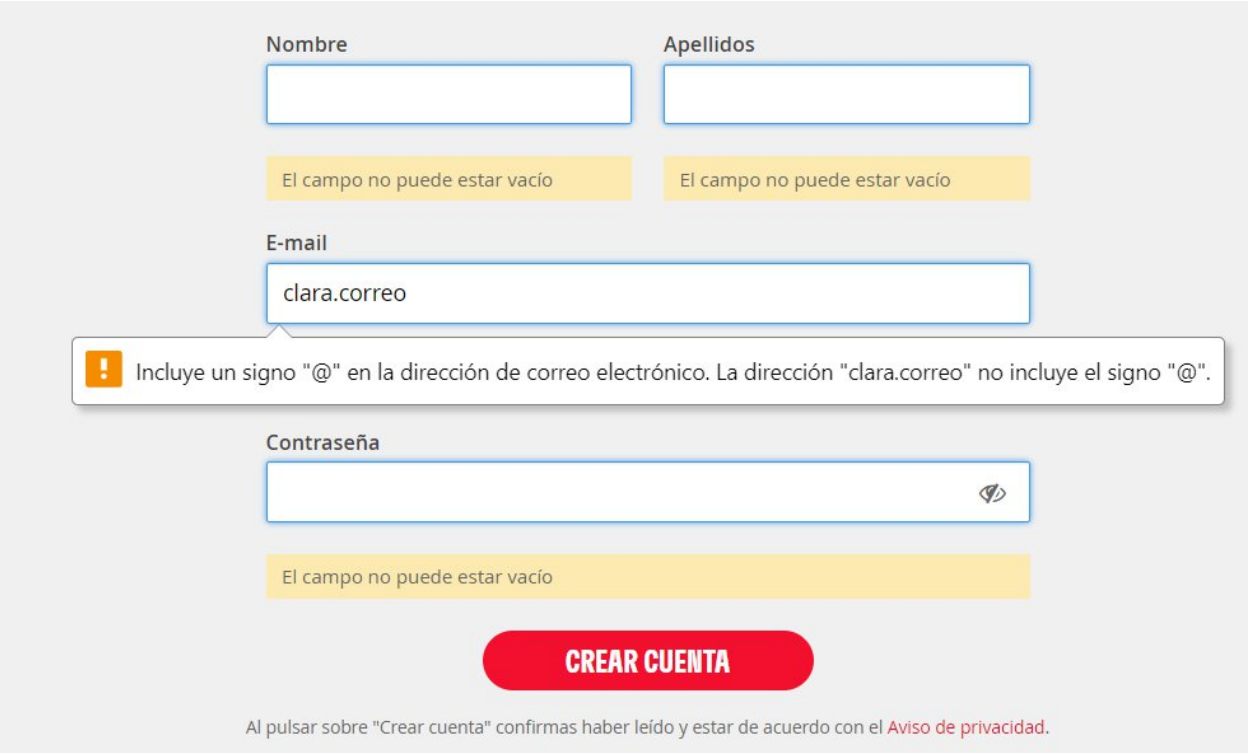
Validación de formularios

En este tema vamos a ver cómo realizar una de las acciones principales de Javascript que es la validación de formularios en el lado cliente.

Antes de enviar los datos al servidor debemos verificar que sean válidos, aunque esta validación NUNCA puede sustituir a la validación en el lado servidor ya que en el lado cliente se puede manipular el código desde la consola para que se salte las validaciones que le pongamos.

Esto se denomina validación de formulario en el lado del cliente y ayuda a garantizar que los datos que se envían coinciden con los requisitos establecidos en los diversos controles de formulario.

Si vamos a cualquier web que incluya por ejemplo un formulario de registro, veremos que si no introducimos los datos obligatorios o en el formato que se espera, al pulsar el botón de envío del formulario, nos saldrán distintos tipos de comentarios:



Esto se llama **validación de formulario**. Cuando introducimos los datos, el navegador y/o el servidor web verifican que estén todos los campos obligatorios y además se verifica que estén en el formato correcto y dentro de las restricciones establecidas por la aplicación. La validación realizada en el navegador se denomina **validación en el lado del cliente**, mientras que la validación realizada en el servidor se denomina **validación en el lado del servidor.**

Si los datos son validados en el lado cliente, es decir, la información está en el formato correcto, la aplicación permitirá que los datos se envíen al servidor y que, por ejemplo, se guarden en una base de datos. Si la información no está en el formato correcto, se dará al usuario un mensaje de error indicando lo que debe corregir, permitiendo volverlo a intentar.

¿Por qué se deben validar los formularios?

Las razones principales por las que se debe validar un formulario son las siguientes:

Obtener los datos correctos y en el formato correcto. Nuestras aplicaciones no funcionarán correctamente si los datos de nuestros usuarios no están completos, son incorrectos o se almacenan en el formato incorrecto.

Proteger los datos de los usuarios. Obligar a un usuario a introducir contraseñas seguras facilita la protección de la información de su cuenta.

Proteger nuestra aplicación. Hay muchas formas en que los usuarios maliciosos puedan usar mal los formularios desprotegidos y dañar la aplicación, por ejemplo con inyección de código SQL.

Tipos de validación en el lado cliente

Encontramos dos tipos de validación en el lado cliente:

- Usar la **validación incorporada en HTML5** y dejar que sea el navegador quien se encargue de todo. Este tipo de validación tiene un mejor rendimiento que JavaScript, pero no es tan personalizable como la validación con JavaScript.
- Realizar nosotros la **validación mediante Javascript**. Esta validación es completamente personalizable, pero debemos crearlo todo.

Validación con HTML5

Una de las principales características de los controles de formulario de HTML5 es la capacidad de validar la mayoría de los datos de usuario sin depender de JavaScript. Esto se realiza mediante el uso de atributos de validación en los elementos del formulario. Los vimos cuando tratamos los formularios, pero recapitulamos aquí:

Atributo	Descripción
`required`	Especifica si un campo de formulario debe completarse antes de que se pueda enviar el formulario.
`min-length` `max-length`	Especifican la longitud mínima y máxima de los datos de texto (cadenas).
`min` `max`	Especifican los valores mínimo y máximo de los tipos de entrada numéricos.
`type`	Especifica si los datos deben ser un número, una dirección de correo electrónico,...
`pattern`	Especifica una expresión regular que define un patrón que los datos que se introduzcan deben seguir.

Si los datos que se introducen en un campo de formulario siguen todas las reglas que especifican los atributos anteriores, se consideran válidos y el formulario será enviado, siempre que no haya nada más que lo impida (por ejemplo, Javascript). Si no, se consideran no válidos y el navegador bloquea el formulario mostrando el mensaje de error.

Cuando un elemento es válido podemos aplicarle un estilo específico utilizando la pseudoclase `:valid` de CSS.

De igual manera, si queremos aplicar un estilo específico a los elementos no válidos, podemos usar la pseudoclase `:invalid` de CSS.

Ejemplo con required

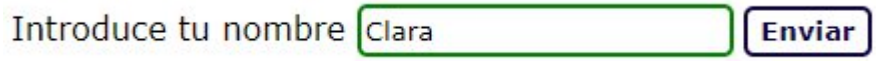
HTML

```
<form>
        <label for="nombre">Introduce tu nombre</label>
        <input type="text" required />
        <button>Enviar</button>
    </form>
```

CSS

```
input[type="text"]
{
    height: 20px;
    border-radius: 5px;
}
input[type="text"]:valid
{
    border:solid 2px  green;
}
input[type="text"]:invalid{
    border:solid 2px red;
}
```

El CSS da un borde verde cuando la entrada es válida (se ha rellenado el campo) o borde rojo cuando la entrada no es válida, es decir, cuando el campo no se ha rellenado



Validación de una expresión regular

Otra característica útil de validación es el atributo `pattern`, que espera una expresión regular como valor. Una expresión regular (regex) es un patrón que se puede usar para establecer combinaciones de caracteres en cadenas de texto, por lo que las expresiones regulares son ideales para la validación de formularios y sirven para una gran variedad de otros usos en JavaScript.

Veamos algunos ejemplos de expresiones regulares:

Expresión	Descripción
a	Debe coincidir con un caracter que es 'a' (no es 'b', ni 'aa')
abc	Debe coincidir con a seguido de b y de c
ab?c	Debe ser 'a' seguida por una 'b' o no, y una 'c' ('abc' o 'ac')
ab*c	Debe ser 'a' seguida opcionalmente por una o varias 'b', y una 'c' ('abc', 'abbbbc' o 'ac')
a|b	O es 'a' o es 'b' ('a' o 'b', pero no 'ab')
abc|xyz	Debe coincidir exactamente con abc o xyz (pero no con 'abcxyz', 'a' o 'y').
[A-Z]{3}[0-9]{4}	Debe coincidir con 3 letras mayúsculas y 4 números.
Más expresiones regulares en: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_expressions https://www.adictosaltrabajo.com/2015/01/29/regexsam/#02	

Validar formularios con Javascript

Veamos ahora cómo validar un formulario con un script en JavaScript. Esto significa que se fijarán unas condiciones para cuando el usuario rellene los diversos campos del formulario. Si las condiciones son las esperadas, el formulario podrá enviarse, de lo contrario, el envío será cancelado.

HTML

```
 <form onsubmit="return validar();" action="Ejemplo2B.html" method="post">
      <label for="nombre">Introduce tu nombre</label>
      <input type="text" id="txtNombre"  />
      <button>Enviar</button>
   </form>
```

JS

```
function validar()
{
   let nombre=document.getElementById("txtNombre");
   if(nombre.value.length>2)
   {
      return true;
   }
   else{
      return false;
   }
}
```

Observemos el manejador de eventos `onsubmit` de la etiqueta `<form>`, desde el que se hace una llamada a la función `validar()`. Esta función tiene como objetivo comprobar que el usuario ha introducido datos en el campo que consideramos obligatorio. Es muy difícil impedir que un usuario nos engañe si quiere, pero podemos delimitar bastante la variedad de cosas válidas que puede escribir en los campos de texto del formulario. Por ejemplo, si ponemos un `required` en el elemento, un usuario avanzado puede borrarlo y no tendría problema en enviar el formulario, pero nos aseguramos que el formulario no se envíe si los datos no son correctos con la función `validar()`